WELT 6 · DOPPELSTUNDE 38 · NACH DER PROBEKLAUSUR

W6·1 – Korrektur und Fehlerjagd

Material: deine Probeklausur „Die Paketstation“, dieses AB, AQ_W6_Probeklausur_Papier-Compiler.txt (Online-IDE) · Im Lab (Algo Quest Lab): Level 6.1 und 6.2 (Fallenjagd I und II), Level 6.3 und 6.4 (Abi-Mix I und II), Online-Boss „Generalprobe online“

Die Generalprobe ist geschlagen – jetzt wird ausgewertet. Profis schauen sich nach dem Spiel die Aufzeichnung an: Wo sind die Punkte liegen geblieben? Heute korrigierst du deine Probeklausur selbst nach den **Kernpunkten** des Erwartungshorizonts, lässt sie von deinem Team gegenprüfen, findet gemeinsam eure häufigsten Fehler und trainierst sie gezielt weg. Am Ende steht dein persönlicher **Abi-Fahrplan** für die Klausur „Ladepark“.

Schritt	Was?	Zeit	Im Lab
1	Selbstkorrektur mit dem Korrekturbogen (Teil 1)	20 min	–
2	Peer-Check im Team: Zweitkorrektur, Abweichungen klären (Teil 2)	15 min	–
3	Fehlerstatistik: deine Fehlerliste und die Team-Strichliste P1 – P10 (Teil 3)	10 min	Profil: „Deine Fallen“
4	Fehlerjagd: Trainingsaufgaben zu euren Top-Fehlern (Teil 4)	25 min	Level 6.1 und 6.2
5	Mein Abi-Fahrplan (Teil 5)	10 min	Level 6.3 und 6.4
6	Papier-Compiler: deine Implementierungen abtippen und testen (Teil 6)	10 min + zu Hause	–

Fallen jagen – auf Papier und im Algo Quest Lab

- **Auf Papier (dieses AB):** Korrektur, Fehlerjagd, Abi-Fahrplan und Papier-Compiler sind Abi-Training – ohne Punkte. Kontrolle der Fehlerjagd mit der Lösung bei der Lehrkraft.
- **Deine Fallen:** Dein Profil im Lab zeigt unter „Deine Fallen“, bei welchen P-Nummern (P1 – P10) du dort Fehlversuche hattest. Vergleiche mit deiner Fehlerliste aus Teil 3 – stimmen deine Top 3 überein?
- **Fallenjagd im Lab:** Level 6.1 „Fallenjagd I – P1 bis P5“ und Level 6.2 „Fallenjagd II – P6 bis P10“ – jede Aufgabe trainiert genau eine Falle und wird automatisch geprüft. **Abi-Mix:** Level 6.3 „Abi-Mix I – Welten 0 bis 2“ und 6.4 „Abi-Mix II – Welten 3 bis 5“ für deinen Abi-Fahrplan.
- **Generalprobe online:** Der Online-Boss von Welt 6 (30 min, Auswertung am Ende, bester Versuch zählt) mischt alle Fallen – ideal nach der Fehlerjagd.
- Punkte, Rang und Abzeichen vergibt das Lab automatisch. Die Klausur „Ladepark“ wird benotet – Lab-Punkte und Note bleiben getrennt.

Teil 1 – Selbstkorrektur mit dem Korrekturbogen

So korrigierst du nach Kernpunkten

- Ein Kernpunkt zählt nur, wenn er in deiner Lösung **vollständig und fachlich richtig** steht – „fast“ oder „gemeint“ zählt nicht.
- Folgefehler: Hast du einen Wert falsch und rechnest damit richtig weiter, verlierst du nur den Kernpunkt mit dem ersten Fehler.
- Hake in der Spalte **selbst** ab, was du erreicht hast. Für jeden **nicht** erreichten Kernpunkt machst du einen Strich bei der angegebenen P-Nummer in deiner Fehlerliste (Teil 3). Stehen mehrere P-Nummern da, entscheide, welche passt.
- Die Spalte **Peer** füllt deine Zweitkorrektorin bzw. dein Zweitkorrektor in Teil 2 aus.

a) Konstruktoren und Kapselung erläutern 4 BE (I 3 · II 1 · III 0)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Konstruktor als besondere Operation: erzeugt ein Objekt der Klasse und belegt seine Attribute mit Anfangswerten	I	1	–	☐	☐
Paketstation(standort): setzt den Standort, erzeugt leere dynamische Reihung, leere Schlange und leeren Stapel – Station ohne Fächer (Fächer später mit fachHinzufuegen)	I	1	P10	☐	☐
Paketstation(standort, anzahlS, anzahlM, anzahlL): erzeugt zusätzlich die angegebene Anzahl Fächer je Größe und hängt sie an (Abb. 2: 2/3/2 → Fächer 101 bis 107); gleicher Name, verschiedene Parameterlisten (Überladung)	I	1	P10	☐	☐
„–“: privat, nur innerhalb von Fach zugreifbar (Kapselung); Zugriff über die öffentliche (+) Operation getInhalt(), danach getNummer() von Paket; vorher istFrei() prüfen (null)	II	1	P10	☐	☐
Summe Teilaufgabe a)		4		___	___

b) Implementierung: Suche in einer dynamischen Reihung von Objekten 7 BE (I 1 · II 4 · III 2)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Methodenkopf passend zur Signatur (Rückgabetyp Ganzzahl, Parameter Zeichenkette)	I	1	P3	☐	☐
Durchlauf aller Fächer mit Index 0 bis faecher.getLength() – 1	II	1	P6	☐	☐
Zugriff mit getItem(i) und Cast (Fach)	II	1	P4	☐	☐
istFrei() wird vor getInhalt() geprüft – kein Zugriff über null	II	1	P1	☐	☐
Vergleich der Sendungsnummern mit equals	II	1	P5	☐	☐
Rückgabe der Fachnummer (getNummer(), nicht der Index i) beim ersten Treffer	III	1	P3	☐	☐
Rückgabe −1 erst nach dem vollständigen Durchlauf; Rückgabe auf jedem Pfad; leere Station ohne Laufzeitfehler	III	1	P3	☐	☐
Summe Teilaufgabe b)		7		___	___

c) Struktogramm: Schlange auswerten und unverändert lassen 6 BE (I 1 · II 4 · III 1)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Kopf mit Signatur, Zählvariable mit 0 initialisiert, Rückgabe mit „gib … zurück“	I	1	P9	☐	☐
Hilfsschlange erzeugen (erzeuge Schlange)	II	1	–	☐	☐
Schleife „solange nicht auftraege.isEmpty()“ mit dequeue – keine Entnahme aus leerer Schlange	II	1	P1	☐	☐
Vergleich p.getGroesse() = groesse (Zeichen) und Zählen	II	1	P4	☐	☐
jedes Paket in die Hilfsschlange einreihen und danach alle Pakete zurück in auftraege	II	1	P2	☐	☐
Reihenfolge bleibt erhalten (Hilfsstruktur Schlange, kein Stapel); Notation nach EH §5 (←, Schleifen, Verzweigung, keine Java-Syntax)	III	1	P2 P9	☐	☐
Summe Teilaufgabe c)		6		___	___

d) Tracetabelle mit Schlange, dynamischer Reihung und Stapel 4 BE (I 2 · II 2 · III 0)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Durchläufe 1 und 2: P-4127 (M) → i = 3, Fach 104; P-9054 (L) → i = 6, Fach 107	I	1	P8	☐	☐
Durchläufe 3 und 4: P-1863 (S) → i = 1, Fach 102; P-6620 (M) → i = 4, Fach 105 (Fach 104 ist seit Durchlauf 1 belegt)	I	1	P8	☐	☐
Durchläufe 5 und 6: kein freies M- bzw. L-Fach, i = 7, gefunden falsch; ruecksendungen P-3089 (M) bzw. P-8845 (L), P-3089 (M) (oben zuerst)	II	1	P8	☐	☐
anzahl in jeder Zeile fortgeschrieben (1, 2, 3, 4, 4, 4), Rückgabe 4; alle vorgegebenen Spalten ausgefüllt	II	1	P8	☐	☐
Summe Teilaufgabe d)		4		___	___

e) Veränderung eines Algorithmus erläutern 2 BE (I 0 · II 1 · III 1)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Ohne Treffer bei gleicher Größe: weitere Suche nach einem freien Fach der nächstgrößeren Größe (S → M → L) bzw. Bedingung „Fachgröße ≥ Paketgröße“; Rücksendung erst, wenn auch diese Suche erfolglos ist	II	1	P10	☐	☐
Kleinstes passendes Fach: Suche größenweise aufsteigend in getrennten Durchläufen (nicht „erstes Fach mit ausreichender Größe“, da die Fächer nicht nach Größe sortiert sein müssen) oder Minimum der Größe über alle passenden freien Fächer bestimmen; Größenvergleich z. B. über eine Hilfsoperation Größe → Volumen	III	1	P10	☐	☐
Summe Teilaufgabe e)		2		___	___

f) Rekursionstabelle und Prinzip der Rekursion 5 BE (I 2 · II 3 · III 0)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Tiefen 0 bis 2 korrekt (Erstaufruf Tiefe 0; s, platz 8, 6, 5; p und Volumen)	I	1	P7 P8	☐	☐
Tiefen 3 und 4 korrekt; Abbruch in Tiefe 4, weil P-1190 (M) nicht passt (2 > 1)	I	1	P7 P8	☐	☐
Rückgaben von innen nach außen 0, 1, 2, 3, 4; Rückgabe des Erstaufrufs 4	II	1	P7	☐	☐
Erläuterung: sammle ruft sich selbst mit einem kleineren Problem auf (Stapel um ein Paket kürzer, platz um dessen Volumen kleiner)	II	1	P10	☐	☐
Basisfälle (leerer Stapel; oberstes Paket passt nicht) beenden die Rekursion mit 0; das Ergebnis entsteht auf dem Rückweg als 1 + Ergebnis des rekursiven Aufrufs	II	1	P7	☐	☐
Summe Teilaufgabe f)		5		___	___

g) Rekursionstiefe begründen 2 BE (I 0 · II 1 · III 1)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Selbstaufruf nur, wenn das oberste Paket passt (getVolumen() ≤ platz); jedes Paket hat ein Volumen von mindestens 1 – platz sinkt mit jeder Rekursionsebene um mindestens 1	II	1	P10	☐	☐
platz wird nie negativ; in Tiefe t ist platz höchstens um t kleiner als beim Erstaufruf → t ≤ platz (Erstaufruf Tiefe 0); Gleichheit nur bei lauter S-Paketen; die Anzahl der Pakete spielt für die Schranke keine Rolle	III	1	P7 P10	☐	☐
Summe Teilaufgabe g)		2		___	___

h) Fehleranalyse einer rekursiven Methode 5 BE (I 1 · II 3 · III 1)

Kernpunkt	AFB	BE	P bei Verlust	selbst	Peer
Fehler 1 (Zeile 10): Ergebnis des rekursiven Aufrufs wird nicht verwendet – es zählt nur das Fach an Index i (Erstaufruf: Fach 101 ist belegt → 0)	I	1	P7	☐	☐
Fehler 2 (Zeile 2): Basisfall schon bei i = getLength() − 1 – das letzte Fach (107, frei, Größe L) wird nie geprüft	II	1	P6 P7	☐	☐
Korrektur: Basisfall i == faecher.getLength() bzw. i >= faecher.getLength() liefert 0; Ergebnis des rekursiven Aufrufs addieren	II	1	P7	☐	☐
korrigierte Methode vollständig und lauffähig (Cast, Bedingung, Rückgabe auf jedem Pfad, rekursiv – keine Schleife)	II	1	P3 P4	☐	☐
Station ohne Fächer: getLength() − 1 = −1, der Basisfall greift für i = 0 nicht → getItem(0) → Laufzeitfehler (IndexOutOfBoundsException); die korrigierte Methode liefert 0	III	1	P1	☐	☐
Summe Teilaufgabe h)		5		___	___

Teil 2 – Peer-Check im Team

Tauscht innerhalb des Teams die Probeklausuren im Ring (A → B → C → A). Die Zweitkorrektur arbeitet mit **deinem** Korrekturbogen und hakt in der Spalte **Peer** ab, **ohne** vorher auf deine Häkchen zu schauen (Häkchen abdecken!). Danach vergleicht ihr. Bei jeder Abweichung entscheidet ihr gemeinsam mit dem Kernpunkt-Text – nicht mit dem Gefühl. Wenn ihr euch nicht einigt: Lehrkraft fragen.

Teil	max.	selbst	Peer	Abweichung?	Einigung und Begründung (Kernpunkt, Stelle in der Lösung)
a)	4			☐ ja ☐ nein	
b)	7			☐ ja ☐ nein	
c)	6			☐ ja ☐ nein	
d)	4			☐ ja ☐ nein	
e)	2			☐ ja ☐ nein	
f)	5			☐ ja ☐ nein	
g)	2			☐ ja ☐ nein	
h)	5			☐ ja ☐ nein	
Σ	35				

Ergebnis nach dem Peer-Check (bei Abweichungen gilt die Einigung)

	a)	b)	c)	d)	e)	f)	g)	h)	Summe
max. BE	4	7	6	4	2	5	2	5	35
selbst									
Peer									
Ergebnis									

Gute Zweitkorrektur

- Lies die Lösung so, wie ein Abi-Korrektor sie liest: Steht es **da**? Nicht: Hätte er es gewusst?
- Code im Kopf ausführen: Was passiert bei einer **leeren** Station, beim **letzten** Fach, wenn das Paket **fehlt**?
- Struktogramm: Ist jede Zeile nach §5 notiert? Steht irgendwo Java (==, ++, ;)?
- Bei „erläutern"/„begründen": Gibt es einen **Bezug zum Material** (Paketstation, Fächer, Abbildungen)?

Korrigiert von: _____ *(Team _____)* *Einverstanden:* _____

Unterschreibt erst, wenn die Zweitkorrektur **vollständig** ist: alle Kernpunkte bearbeitet, alle Abweichungen begründet.

Teil 3 – Fehlerstatistik: Die Top 10 Punktverluste

Die meisten Punkte gehen im Abitur nicht durch fehlendes Wissen verloren, sondern durch immer dieselben **zehn Fehlertypen**. Jede P-Nummer steht im Korrekturbogen neben dem Kernpunkt, bei dem dieser Fehler typischerweise Punkte kostet.

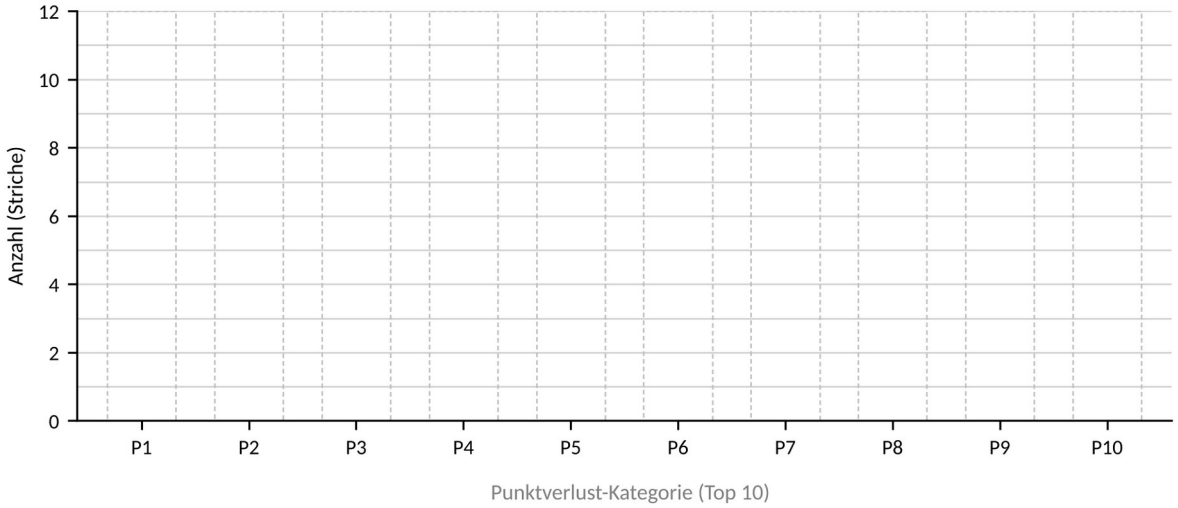
P	Punktverlust	Woran du ihn erkennst
P1	Laufzeitfehler nicht ausgeschlossen	isEmpty vor top/pop/head/dequeue/getItem/getLeft; Indexgrenzen 0 … getLength() − 1
P2	Struktur zerstört statt unverändert	Stapel/Schlange nach dem Durchlauf nicht wiederhergestellt, obwohl gefordert
P3	Signatur verfehlt	Rückgabetyp, Parameter, Name, static/Sichtbarkeit; Rückgabe fehlt auf einem Pfad
P4	Typen und Casts	(Integer) vergessen, Zeichen vs. Zeichenkette, ganzzahlige Division
P5	Zeichenketten mit == statt equals	Zeichenketten mit == statt equals; nicht erlaubte String-Operationen
P6	Off-by-one	< statt <=, getLength() statt getLength() − 1, Startindex
P7	Rekursion	Basisfall fehlt/falsch, Ergebnis des rekursiven Aufrufs nicht verwendet, Rekursionstiefe falsch (Erstaufruf = 0)
P8	Trace-/Dokumentationstabelle	nur Variablen, keine Bedingungen; eine Zeile an jeder markierten Stelle („// → Tabellenzeile“); vorgegebene Spalten, Endwerte
P9	Struktogramm nicht nach EH §5	←, „gib … zurück“, Schleifenarten, Fallunterscheidung; Java-Syntax im Struktogramm
P10	Operator verfehlt	„erläutern“ als bloße Nennung, „begründen“ ohne Begründung, kein Bezug zum Material, Fachbegriffe fehlen

a) Meine Fehlerliste und die Team-Strichliste. Trage zuerst deine Striche aus Teil 1 in die Spalte „ich“ ein. Sammelt dann die Striche aller Teammitglieder und bildet die Summe.

P	Kurztitel	ich	Mitglied 2	Mitglied 3	Mitglied 4	Team Σ	Warm-up-Nachtest Σ
P1	Laufzeitfehler nicht ausgeschlossen						
P2	Struktur zerstört statt unverändert						
P3	Signatur verfehlt						
P4	Typen und Casts						
P5	Zeichenketten mit == statt equals						
P6	Off-by-one						
P7	Rekursion						
P8	Trace-/Dokumentationstabelle						
P9	Struktogramm nicht nach EH §5						
P10	Operator verfehlt						

Die letzte Spalte füllt ihr in DS 39 nach dem Warm-up-Nachtest aus: Bei welchen P-Nummern ist euer Team besser geworden (Team Σ minus Warm-up-Nachtest Σ)? Die Lehrkraft fasst alle Teams zur Kursstatistik zusammen; im Lab zeigt dein Profil unter „Deine Fallen“ deine eigenen Fehlversuche je P-Nummer.

b) Diagramm. Zeichnet die Teamsummen als Säulendiagramm (ein Kästchen = ein Strich). Markiert eure **Top 3** farbig – genau diese Aufgaben bearbeitet ihr in Teil 4 zuerst.



Unsere Top 3: **1. P __** ***2. P*** ***3. P __***

Teil 4 – Fehlerjagd: P1 bis P10

Zu jedem der zehn Punktverluste gibt es eine kurze Trainingsaufgabe im Abi-Stil (2 bis 5 Minuten, **ohne Gerät**). Beginnt mit euren Top 3, teilt die übrigen im Team auf und erklärt sie euch gegenseitig. Alle Aufgaben spielen in der Paketstation aus der Probeklausur (Klassen Paket, Fach, Paketstation). Weitere Aufgaben zu jeder Falle gibt es im Lab (Level 6.1 und 6.2).

P1: Absturz verhindern

PAPIER ★

Beide Methoden der Klasse Paketstation können abstürzen.

```
// Sendungsnummer der obersten Ruecksendung; "" wenn es keine gibt
public String obersteRuecksendung() {
    Paket p = (Paket) ruecksendungen.top();
    return p.getNummer();
}

// Fachnummer des Fachs an Position index; -1 bei ungueltigem Index
public int fachnummerAn(int index) {
    Fach f = (Fach) faecher.getItem(index);
    return f.getNummer();
}
```

a) Gib für jede Methode einen Aufruf (für eine passende Station) an, der zu einem Laufzeitfehler führt, und benenne die Ursache.

b) Ergänze die nötigen Abfragen. Schreibe beide Methoden vollständig neu auf dein Papier.

P2: Der verschwundene Stapel

PAPIER ★★

```
// Anzahl der L-Pakete im Stapel s; s soll danach UNVERAENDERT sein
public int zaehleL(Stack s) {
    int anzahl = 0;
    Stack hilf = new Stack();
    while (!s.isEmpty()) {
        Paket p = (Paket) s.pop();
        if (p.getGroesse() == 'L') {
            anzahl++;
        }
        hilf.push(p);
    }
    s = hilf;
    return anzahl;
}
```

a) zaehleL wird mit dem Stapel s aus Abbildung 6 der Probeklausur aufgerufen (oben zuerst: P-2741 (M), P-0412 (S), P-7781 (L), P-5236 (S), P-1190 (M)). Gib die Rückgabe und den Inhalt von s nach dem Aufruf an.

b) Erläutere, warum die Zeile `s = hilf;` den Stapel nicht wiederherstellt – selbst wenn hilf dieselben Pakete in derselben Reihenfolge enthielte.

c) Korrigiere die Methode (nur die geänderten Zeilen).

P3: Signatur-Check

PAPIER ★

Vorgabe: Die Operation **istVoll(groesse: Zeichen): Wahrheitswert** der Klasse Paketstation liefert wahr, wenn kein freies Fach der Größe groesse vorhanden ist. Eine Schülerin schreibt:

```
public void istVoll(String groesse) {
    for (int i = 0; i < faecher.getLength(); i++) {
        Fach f = (Fach) faecher.getItem(i);
        if (f.istFrei() && f.getGroesse() == groesse) {
            return false;
        }
    }
}
```

a) Finde die **drei** Abweichungen von der Vorgabe bzw. Stellen, an denen der Compiler meckert.

b) Schreibe die Methode korrekt auf dein Papier.

P4: Ganz schön ganzzahlig

PAPIER ★★

```
// durchschnittliches Volumen der Auftraege; 0.0 bei leerer Schlange;
// die Schlange auftraege bleibt unveraendert
public double durchschnittVolumen() {
    int summe = 0;
    int anzahl = 0;
    Queue hilf = new Queue();
    while (!auftraege.isEmpty()) {
        Paket p = auftraege.dequeue();
        summe = summe + p.getVolumen();
        anzahl++;
        hilf.enqueue(p);
    }
    while (!hilf.isEmpty()) {
        auftraege.enqueue(hilf.dequeue());
    }
    if (anzahl == 0) {
        return 0.0;
    }
    return summe / anzahl;
}
```

a) Die Methode kompiliert nicht. Gib die Zeile und den Grund an.

b) Nach der Korrektur von a) liefert die Methode für die sechs Zustellaufträge aus Abbildung 2 der Probeklausur einen falschen Wert. Berechne ihn und den richtigen Durchschnitt.

c) Korrigiere die letzte Zeile.

P5: Gleich ist nicht gleich

PAPIER ★

```
// Anzahl der eingelagerten Pakete fuer den Empfaenger name
public int zaehleEmpfaenger(String name) {
    int anzahl = 0;
    for (int i = 0; i < faecher.getLength(); i++) {
        Fach f = (Fach) faecher.getItem(i);
        if (!f.istFrei() && f.getInhalt().getEmpfaenger() == name) {
            anzahl++;
        }
    }
    return anzahl;
}
```

a) Markiere den Fehler und korrigiere die Zeile.

b) Im Papier-Compiler laufen alle Tests trotzdem grün. Erkläre, warum – und warum du im Abitur trotzdem Punkte verlierst.

c) Kreuze an, welche Zeichenketten-Operationen im Abitur erlaubt sind (Ergänzende Hinweise):

☐ length() ☐ charAt(i) ☐ toUpperCase() ☐ substring(a, b) ☐ indexOf(c) ☐ contains(t) ☐ split(" ") ☐ compareTo(t) ☐ trim() ☐ equals(t)

P6: Eins daneben

PAPIER ★★

```
// Fachnummer des LETZTEN freien Fachs; -1, wenn kein Fach frei ist
public int letztesFreiesFach() {
    for (int i = faecher.getLength(); i > 0; i--) {
        Fach f = (Fach) faecher.getItem(i);
        if (f.istFrei()) {
            return f.getNummer();
        }
    }
    return -1;
}
```

a) Gib an, was beim Aufruf für die Station aus Abbildung 2 der Probeklausur passiert.

b) Jemand ändert nur den Startwert auf `faecher.getLength() - 1`. Für eine Station, in der nur Fach 101 frei ist, liefert die Methode trotzdem −1. Erläutere.

c) Schreibe den richtigen Schleifenkopf.

P7: Tiefgang

PAPIER ★★

Die Station „Klein“ hat drei Fächer: Index 0 = Fach 101 (S) mit P-7310 (S), Index 1 = Fach 102 (S) frei, Index 2 = Fach 103 (M) mit P-2208 (M).

```
1  // Summe der Volumen aller eingelagerten Pakete ab Index i
2  public int volumenAb(int i) {
3      if (i == faecher.getLength()) {
4          return 0;
5      }
6      Fach f = (Fach) faecher.getItem(i);
7      if (f.istFrei()) {
8          return volumenAb(i + 1);
9      }
10     return f.getInhalt().getVolumen() + volumenAb(i + 1);
11 }
```

a) Dokumentiere den Aufruf `volumenAb(0)` in der Rekursionstabelle.

Rekursionstiefe *)	i	Fach an Index i (Nummer, Inhalt)	Rückgabe
0 (Erstaufruf)	0		

**) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Der Erstaufruf hat die Rekursionstiefe 0.*

b) Gib die Anzahl der Aufrufe und die größte Rekursionstiefe an.

c) In Zeile 2 steht nun `i == faecher.getLength() - 1`. Gib das Ergebnis von volumenAb(0) an und erläutere.

P8: Trace-Detektiv

PAPIER ★★

Die Reihung tage enthält die Lagertage der Pakete einer Station: tage = {3, 7, 7, 2, 9}. Lena dokumentiert den folgenden Ausschnitt mit einer Tracetabelle (nur Variablen): je eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle – vor der Schleife (Startwerte), am Ende jedes Schleifendurchlaufs und nach der Schleife (Endwerte).

```
int max = 0;
int pos = -1;   // → Tabellenzeile (Startwerte)
for (int i = 0; i < tage.length; i++) {
    if (tage[i] > max) {
        max = tage[i];
        pos = i;
    }   // → Tabellenzeile
}   // → Tabellenzeile (Endwerte)
```

Lenas Tabelle:

i	tage[i]	max	pos
–	–	0	−1
0	3	3	0
1	7	7	1
2	7	7	2
4	9	9	4

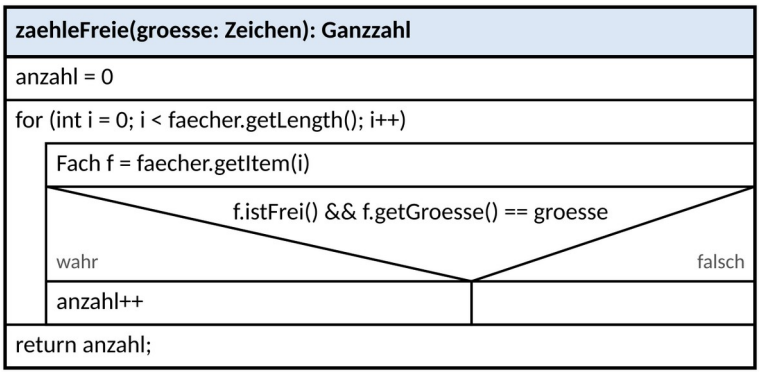
a) Finde die **drei** Fehler in Lenas Tabelle.

b) Schreibe die vollständige, richtige Tabelle auf dein Papier.

P9: Notations-Polizei

PAPIER ★

Das Struktogramm soll die Anzahl der freien Fächer der Größe groesse liefern. Inhaltlich stimmt es – aber die Notation würde im Abitur Punkte kosten.



a) Markiere im Struktogramm alle Stellen, die nicht den Ergänzenden Hinweisen (§5) entsprechen (mindestens fünf).

b) Zeichne das Struktogramm korrekt auf dein Papier.

P10: Operator-Training

PAPIER ★★

Aufgabe: „Begründen Sie, dass für die Zustellaufträge der Paketstation eine Schlange geeignet ist.“ [2 BE] – drei Antworten:

	Antwort	BE (0–2)
A	Eine Schlange arbeitet nach dem FIFO-Prinzip.	
B	Weil man bei einer Schlange nur vorne herausnehmen und hinten anstellen kann, ist das besser als ein Stapel.	
C	Die Aufträge sollen in der Reihenfolge ihres Eintreffens bearbeitet werden. dequeue liefert immer den Auftrag, der am längsten wartet, neue Aufträge werden mit enqueue hinten angestellt (FIFO). So wird kein Auftrag übergangen; den beliebigen Zugriff einer dynamischen Reihung braucht man dafür nicht.	

a) Vergib für jede Antwort 0, 1 oder 2 BE und begründe kurz mit dem Operator „begründen“.

b) Formuliere für Teilaufgabe a) der Probeklausur eine Antwort zum zweiten Konstruktor von Paketstation, die „erläutern“ erfüllt (höchstens drei Sätze, mit Bezug zum Material).

🎯 **Im Lab üben: Level 6.1** „Fallenjagd I – P1 bis P5“ und **Level 6.2** „Fallenjagd II – P6 bis P10“ – Punkte gibt es dort automatisch.

Teil 5 – Mein Abi-Fahrplan

Dein Fahrplan für die Klausur „Ladepark“ (60 BE, 180 min) und für das Abitur. Fülle ihn ehrlich aus – er gehört nur dir.

1. Meine Top-3-Fehler und meine Gegenmaßnahmen

	P-Nr. und Kurztitel	So ist es mir in der Probeklausur passiert	Meine Gegenmaßnahme (konkret!)	Training (Warm-up-Karten, Lab-Level, AB, Video) bis …
1				
2				
3				

Beispiel für eine konkrete Gegenmaßnahme: „Vor jedem getItem schreibe ich die Indexgrenze an den Rand“ statt „besser aufpassen“.

2. Mein Zeitplan (≈ 2,5 Minuten je BE)

BE einer Teilaufgabe	2	3	4	5	6	8	10	60 (Klausur)
Zielzeit (min)	5	7,5	10	12,5	15	20	25	150

180 min Klausur − 150 min für 60 BE = 30 min für Einlesen, Planen und Prüfen. In der Probeklausur habe ich __ *min gebraucht und min Reserve gehabt. Am meisten Zeit hat mich Teilaufgabe gekostet, weil* _____.

3. Mein Vorgehen in der Prüfung (Reihenfolge festlegen, Häkchen setzen, wenn du es in der Klausur umgesetzt hast)

Phase	Was ich tue	✔
Einlesen (10–15 min)	Alle Aufgaben **überfliegen**; Operatoren unterstreichen; im Material Klassenkarten, Signaturen und Beispielwerte **markieren**.	☐
BE-Plan (2 min)	Zielzeiten an den Rand schreiben (2,5 min je BE); Reihenfolge festlegen: _____	☐
Bearbeiten	Bei Implementierungen zuerst den Methodenkopf aus der Signatur abschreiben; Tabellen mit allen vorgegebenen Spalten; hängen bleiben → markieren, weiter, später zurück.	☐
Abi-Check (je Methode)	Leer? Index? Cast? equals? return auf jedem Pfad? Struktur unverändert? Basisfall? Struktogramm ohne Java?	☐
Reserve (10–15 min)	Markierte Lücken schließen, „erläutern/begründen“-Antworten auf Bezug zum Material prüfen.	☐

4. Mein persönlicher Abi-Check (ergänze deine eigenen Prüffragen zu deinen Top-3-Fehlern)

🎯 **Im Lab üben: Level 6.3** „Abi-Mix I – Welten 0 bis 2“ und **Level 6.4** „Abi-Mix II – Welten 3 bis 5“ – Punkte gibt es dort automatisch. Zum Abschluss: der Online-Boss **„Generalprobe online“** (30 min, bester Versuch zählt).

Teil 6 – Papier-Compiler: Läuft deine Handschrift?

Regeln des Papier-Compilers

1. Öffne in der Online-IDE den Workspace **„W6 Papier-Compiler“** (Datei AQ_W6_Probeklausur_Papier-Compiler.txt) und prüfe, ob die Bibliothek „Niedersachsen“ aktiviert ist (Rechtsklick auf den Workspace → Einstellungen).
2. Tippe deine Lösung zu **b) sucheFach** Zeichen für Zeichen **unverändert** in den Abschnitt „DEIN TEIL“ ab – auch mit Fehlern! Kür: c) dein Struktogramm Block für Block in Java übersetzen, h) deine Korrektur abtippen.
3. Reiter „Testrunner“ → grüner Pfeil neben ProbeklausurTest. Tests aufgabeB_, aufgabeC_, aufgabeH_. Notiere das Ergebnis des **ersten** Laufs, bevor du etwas änderst.
4. Jeden Fehler ins Fehlerlogbuch – erst dann korrigieren, bis alles grün ist.
5. Achtung: == statt equals (P5) und eine Schleife statt Rekursion in h) finden die Tests **nicht** – das prüfst du selbst mit dem Korrekturbogen.

Teilaufgabe	Tests	1. Lauf: grün / gesamt	nach Korrektur	Ziel
b) sucheFach (Pflicht)	aufgabeB_ (6)	___ / 6	___ / 6	alle grün im 1. Lauf
c) anzahlAuftraege (Kür)	aufgabeC_ (5)	___ / 5	___ / 5	alle grün im 1. Lauf
h) freieFaecher (Kür)	aufgabeH_ (6)	___ / 6	___ / 6	alle grün im 1. Lauf

Fehlerlogbuch

Nr.	Was war falsch? (Zeile, Meldung)	P-Nr.	Art: Syntax / Logik / Abi-Check	So vermeide ich es
1				
2				
3				
4				